

Interview prep

↳ Not about teaching a concept
About how to answer a question

✓ LLM Foundation

✓ prompt Engg

✓ Embeddings

LLM Foundation

Impact Analytics builds AI systems for retail and consumer goods clients. You've built an LLM-powered assistant that generates category-level demand forecasts as natural-language briefs for merchandisers. Same query, same product, same region — you're getting slightly different briefs on different runs in the same week. A senior merchandiser flags it: **"This isn't trustworthy. Why is the answer changing?"**

Interviewer's question: Explain what's happening and how you'd fix it.

✓
 L_1

✓
 L_2

✓
 L_3

Model Answers – Scenario 1.1

Explaining why outputs change & how we'd fix it

L1 — BARE PASS



“ LLMs are non-deterministic because they use temperature to add randomness. If I set temperature to 0, the outputs will be more consistent. ”



Why L1?

Names the concept but doesn't explain the mechanics.

An interviewer will probe further and if the candidate can't go deeper, they fall out here.

L2 — COMPETENT HIRE



“ LLMs generate tokens by sampling from a probability distribution over the vocabulary. Temperature scales the logits before softmax — higher temperature flattens the distribution and increases randomness; lower temperature sharpens it. Top-p and top-k also affect this by truncating the distribution before sampling. ”

- For a forecast brief where consistency matters, I'd set temperature to 0 or very low, top-p to 1.
- I'd also fix the model version — because if the vendor silently upgrades the model, outputs change even at temp 0.
- If we still see variance, I'd cache the response by input hash so identical queries return the same brief for a stable time window, say 24 hours. ”



Why L2?

Explains mechanism, gives a concrete fix, mentions the caching pattern. This is a hire.

L3 — SENIOR STANDOUT



“ Everything L2 said, plus: temperature 0 does not guarantee determinism in practice. There are three reasons. ”

- 1 **Floating-point non-associativity** — GPU parallel reduction can give slightly different logits across runs, which changes the argmax in tie cases.
- 2 **Most providers batch requests for throughput.** Batching order affects numerical precision, so the same prompt in different batch positions can produce different tokens.
- 3 **Providers sometimes route to different model replicas or hardware,** and model updates roll out silently.

So the real answer is: use a seed parameter if the provider exposes one — OpenAI has seed, Anthropic doesn't as of now. Pin the model version. Cache aggressively. And be honest with the merchandiser that 'the same input gives the same output' is a distributed systems guarantee, not an LLM guarantee. If bit-exact consistency is a compliance requirement, we should not use an LLM for the final generation — we should use it to fill a structured template. ”



Why L3?

Shows the candidate has actually run into this in production. The “use LLM to fill a template” insight is what separates staff engineers from mid-levels.



Key Takeaway

Control randomness, lock down the environment, and use caching. Understand that true determinism is not guaranteed.

FOLLOW-UPS

Q: What's the difference between temperature and top-p, and can I use both?

Temperature reshapes the full probability distribution by dividing logits before softmax. Top-p (nucleus sampling) truncates the distribution to the smallest set of tokens whose cumulative probability exceeds p, then samples from that set. Yes, you can use both — top-p first, temperature scales what remains. Most production systems use one or the other. My default: temperature 0.2, top-p 1 for structured tasks; temperature 0.7, top-p 0.9 for creative tasks.

Q: You set temperature to 0. Now the merchandiser says the answers are "robotic and repetitive." What do you do?

That's the trade-off. If we need creativity and consistency both, we don't get them from one setting. Options: use temperature 0 for the structured facts and a small creative rewrite pass at higher temp for the prose layer; or generate at low temp, cache, and vary presentation via templates.

Q: What's a seed? Does OpenAI's seed guarantee determinism?

Seed initializes the random state for sampling. OpenAI's seed parameter is best-effort — they document it as "mostly deterministic" but not guaranteed. They also return a `system_fingerprint` that identifies the backend config; if that changes, outputs may change even at the same seed.

Token Math for API Cost

OptimizeGEO is building a document Q&A service on top of GPT-4. A user uploads a 20-page PDF and asks a question. Product manager asks you: **"How much does one query cost us, and how fast is the response?"**

Interviewer's question: Walk me through your reasoning - tokens, cost, latency.



L1 — BARE PASS



“

20 pages is a lot of text, so it will cost more. Cost depends on the model — GPT-4 is expensive. Latency will be slow because there's a lot to process.



Why L1?

No numbers. No token estimation.

An interviewer will not extend an offer to someone who can't reason about cost quantitatively — that's the job.

L2 — COMPETENT HIRE



Token Estimation

- Rule of thumb: 1 token $\approx$ 4 characters or 3/4 of a word (English)
- A page of dense text $\approx$ 500 words $\approx$ 650 tokens
- 20 pages $\approx$ 13,000 tokens
- + User question $\sim$ 50 tokens
- + System prompt $\sim$ 200 tokens
- + Context / instructions $\sim$ 300 tokens

➔ Input $\approx$ 13,550 tokens



Output Estimate

- Answer paragraph $\sim$ 200 tokens



Cost Calculation (GPT-4 Turbo)

- $\sim$ \$10 / 1M input tokens $\rightarrow 13,550 / 1M \times \$10 = \$0.1355$ ($\sim$ 13.6¢)
- $\sim$ \$30 / 1M output tokens $\rightarrow 200 / 1M \times \$30 = \$0.006$ ($\sim$ 0.6¢)

➔ Total $\approx$ \$0.14 (14 cents) per query



Latency Estimate

- Prefill (input processed in parallel): $\sim$ 1–2 sec
- Decode (output $\sim$ 50 tokens/sec): 200 tokens $\approx$ 4 sec
- Total end-to-end: $\sim$ 5–6 seconds



Business Impact

- 10,000 queries/day $\rightarrow$ \$1,400/day
- $\$1,400 \times 365 \approx \$500K/\text{year}$

➔ Recommendation: Use RAG instead of stuffing the whole PDF.

L3 — SENIOR STANDOUT



Language Matters

- Token counts vary by language.
- Hindi, Japanese, Chinese are 2–3x more tokens per word than English (due to subword tokenization).
- Use language-adjusted multipliers for cost estimation.



Latency: Prefill vs Decode

- Latency has two components:
 - Prefill: processing the prompt (scales roughly linearly with input length, compute-bound)
 - Decode: generating output (memory-bandwidth-bound)
- Long prompts don't slow down decode speed, but they consume context window and increase prefill time.
- For $\sim$ 13K-token prompt: prefill on GPT-4 $\approx$ 1–2 sec; on some smaller models it can be 5+ sec.



Cost Optimization Order

- 1 Use a cheaper model (e.g., GPT-4o-mini or Claude Haiku) for routing / extraction; use expensive model only for final synthesis.
- 2 Use RAG to bring the input under 2K tokens.
- 3 Use prompt caching — OpenAI & Anthropic offer 50–90% discount on cached prefix tokens. If system prompt is 500 tokens and shared across users, cache it to save input cost per query.



Key Takeaway

Do the math. Estimate tokens, cost, and latency. At scale, optimize with the right model, RAG, and caching.

FOLLOW-UPS

Q: The user pastes the same 20-page PDF for their next 10 questions. How do you reduce cost?

Prompt caching. Cache the system prompt + document prefix; only the trailing user question changes. Cached tokens are 50–90% cheaper. Alternative: parse the PDF once, embed chunks, RAG retrieval brings each query to 2K tokens.

Q: Client says users are complaining that responses take 6 seconds. How do you reduce latency?

Stream tokens as they generate — perceived latency drops from 6 sec to ~1 sec (time-to-first-token). Move to a faster model tier for the extract pass. Reduce output length by prompting for concise answers. Consider parallel model calls if the query has multiple sub-questions.

Q: How would you actually count tokens without calling the API?

Use the tokenizer library — tiktoken for OpenAI models, anthropic.tokenize or the Anthropic SDK for Claude. Never estimate in production — use the real tokenizer for the model you're targeting.

$$\begin{cases} \theta_1 \rightarrow \$0.10 \checkmark \\ \theta_2 \rightarrow \$0.30 \checkmark \end{cases}$$

$$TC: \$0.40$$

Explain the difference between GPT, BERT, and T5. Which one would you pick for building a customer support chatbot?



Model Answers – Scenario 1.3

GPT vs BERT vs T5 — Which for a Customer Support Chatbot?



Interview Question:

Explain the difference between GPT, BERT, and T5.
Which one would you pick for building a customer support chatbot?



Trap Alert:

This looks like a definitions question.
It's actually a systems thinking question.

L1 — BARE PASS



“

GPT is decoder-only, used for generation.
BERT is encoder-only, used for understanding.
T5 is encoder-decoder, used for text-to-text tasks.
For a chatbot I'd use GPT because we need to generate responses. ”



Why L1?

Textbook correct, but zero depth.
Won't get rejected — but won't stand out.

L2 — COMPETENT HIRE



“

- **GPT-family** models are decoder-only, trained with causal language modeling — predict the next token given previous tokens. Great for open-ended generation.
- **BERT** is encoder-only, trained with masked language modeling — predict a masked token given both left and right context. Great for classification, NER, embeddings.
- **T5** is encoder-decoder — reframes every task as text-to-text.

For a customer support chatbot, the answer depends on the layer:



Understanding intent (classification):

A fine-tuned BERT-style model like DistilBERT or a small classifier.



Generating the response:

A GPT-family model.



Semantic search over the knowledge base:

A BERT-family embedding model like BGE or E5.



Honest answer:

It's not one or the other — it's a system that uses each where it's strong.
Most production chatbots use GPT-family for generation and BERT-family for retrieval embeddings.

L3 — SENIOR STANDOUT



“

- Everything L2 said, plus:
- The BERT vs GPT distinction is increasingly less relevant in 2026. Modern general-purpose LLMs are all decoder-only causal models — GPT-4, Claude, Gemini, Llama — because scaling laws favor decoder-only for large-scale pretraining.
Encoder-only models still dominate for embeddings and specific classification tasks because they're small, fast, and bidirectional context helps.
- **T5** is architecturally still around but the T5-style text-to-text framing is now done with decoder-only models via prompting.
- The real modern trade-off isn't GPT vs BERT vs T5. It's:



Proprietary API (OpenAI / Anthropic)

Best quality, managed, but costly.



Open-weight (Llama, Mistral, Qwen)

Flexible, controllable, lower cost to run.



Specialized fine-tuned (small model tuned for your task)

Best for narrow, high-volume tasks.



For a customer support chatbot:

I'd start with a proprietary API for generation quality, use BGE-large for embeddings (open & cheap), and evaluate whether a fine-tuned smaller model can replace the API call once we have production traffic.



Key Takeaway

Don't pick a single model. Build a system that uses the right model at each layer.
Understand architecture, training objective, strengths, and modern trade-offs.

Prompt Engineering

Impact Analytics has a retail client with 50,000 customer complaint tickets. Merchandising team wants to categorize each into one of 12 issue types — sizing, quality, delivery, pricing, and so on — so they can prioritize product changes.

Interviewer's question: You have an LLM API and no training data. Walk me through your prompting strategy from simplest to most advanced. Which do you ship?

Model Answers – Scenario 2.1

Choosing a Prompting Technique for Retail Complaint Classification



Context:

Impact Analytics has a retail client with 50,000 customer complaint tickets. Merchandising team wants to categorize each into one of 12 issue types — sizing, quality, delivery, pricing, and so on.



Interview Question:

You have an LLM API and no training data. Walk me through your prompting strategy from simplest to most advanced. Which do you ship?

L1 — BARE PASS



“

I'd write a prompt asking the LLM to classify each complaint into one of the 12 categories.

Zero-shot. If that doesn't work well,

I'd give examples. ”



Why L1?

Correct but skips the technique names and the trade-offs.

L2 — COMPETENT HIRE



“ I'd walk through four levels.

- 1 **Zero-shot:** “Classify this complaint into one of: sizing, quality, ...” — fast to build, no data needed. Accuracy usually 70–80% for a clean taxonomy.
- 2 **Few-shot:** add 3–5 examples per category in the prompt. Accuracy jumps to 85–90%. Downside: token cost per call scales with example count.
- 3 **Chain-of-thought (CoT):** “Think step-by-step, then output the category.” Better on ambiguous complaints where reasoning helps. Slower and more expensive.
- 4 **Few-shot + CoT combined:** examples that include reasoning traces.



For 50K tickets I'd ship **few-shot without CoT** because latency and cost matter at that volume. I'd hold out 500 tickets, label them manually, use them as an eval set. If accuracy is under 90%, I'd try adding CoT for the ambiguous subset only using a **router: cheap zero-shot for confident cases, CoT for low-confidence ones.**

L3 — SENIOR STANDOUT



“ Everything L2 said, plus:



Question the taxonomy:

Before I write the prompt, I'd ask the client: are the 12 categories actually mutually exclusive? In real retail complaints they're not — “shirt shrunk after wash” is both quality and sizing. My output should be primary+secondary label, or top-k with confidence.



Use structured output with confidence:

Pydantic schema or JSON mode — with a confidence score. Auto-approve high-confidence, route low-confidence to a human reviewer. That's the real production pattern.



Technique nuance:

Self-consistency (sample 5 times at temp 0.7, majority vote) beats CoT alone on hard cases but 5x cost. Not worth it for this classification at this scale.



Think beyond prompting:

For 50K tickets, I'd benchmark a fine-tuned smaller model vs API. Fine-tuning a DistilBERT classifier on first 5K auto-labeled tickets can **cut cost by ~100x** and often beat the LLM on accuracy for a fixed taxonomy. LLM API is right for v1; small fine-tuned model is often the right v2.



Key Takeaway



Start simple, then increase sophistication based on accuracy needs.



Balance accuracy, latency, and cost — not just what's possible.



Think about taxonomy, confidence, evaluation, and when to move to smaller fine-tuned models.

FOLLOW-UPS

Q: What's the ideal number of few-shot examples?

Depends on task complexity and context window budget. Rule of thumb: 3–5 for simple tasks, 5–10 for nuanced ones. Diminishing returns after 8. If you need 20+ examples, you probably need fine-tuning, not prompting.

Q: Order of examples matters. Why?

LLMs have position bias — they attend more strongly to recent tokens. In few-shot, the last example often influences the output more than the first. Best practice: put diverse examples last; keep class distribution balanced; for classification tasks, randomize example order across calls or the model can pattern-match on class order.

Q: When does CoT hurt performance?

Small models — under ~7B params — often can't reason through CoT and their outputs get worse when forced to think step-by-step. CoT is a large-model phenomenon. Also, on simple pattern-matching tasks (like extraction) CoT wastes tokens with no accuracy gain.

gpt-4x → small model (prompt — CoT)

An analysts write market research reports. You're building a pipeline that reads new reports and extracts structured metadata: industry taxonomy code, geographies mentioned, companies mentioned with their role (competitor/partner/supplier), and key financial numbers with units.

Interviewer's question: How do you guarantee valid, parseable output every single time from an LLM? What happens when it fails?

Model Answers – Scenario 2.2

Structured Output at Scale (Euromonitor)



Context:

Euromonitor’s analysts write market research reports. You’re building a pipeline that reads new reports and extracts structured metadata: industry taxonomy code, geographies mentioned, companies mentioned with their role (competitor/partner/supplier), and key financial numbers with units.



Interview Question:

How do you guarantee valid, parseable output every single time from an LLM? What happens when it fails?

L1 — BARE PASS



“

I’d ask the LLM to return JSON in the prompt.

I’d use `json.loads` and if it fails, retry.”



Why L1?

Names the surface approach.

Any interviewer will probe: what specifically do you do when it fails? Does it fail 1% of the time or 20%?

L2 — COMPETENT HIRE



“

Three-layer approach.

1

Schema enforcement at API level: OpenAI has structured outputs with strict JSON schema — the model is constrained at decode time to only emit tokens that match the schema. Anthropic’s tool use similarly forces valid JSON. Failure rate drops to near-zero.

2

Pydantic on the client side: even with API-level enforcement, I validate against a Pydantic model. Pydantic gives type coercion, validation errors, and enum checks. If the API drifts or I use a model without strict mode, this catches it.

3

Retry logic on validation failure: if Pydantic raises, I feed the error back into a follow-up prompt: “Your previous output failed validation with error X. Retry.” Usually succeeds on retry 2.



For Euromonitor’s use case:

- Taxonomy code should be an enum in the schema, not a free-text field.
- Geographies should be restricted to ISO country codes.

→ This eliminates whole classes of hallucination.

L3 — SENIOR STANDOUT



“

Everything L2 said, plus:



4

Business-rule validation (content correctness):

Structured output doesn’t guarantee the content is correct, only the shape. So layer four is business-rule validation: cross-check the extracted geographies against the geographies mentioned in the abstract; if a company is tagged as a competitor, does its name appear in a competitor context sentence?



5

Performance & latency trade-off:

For high-volume extraction, JSON mode has a subtle cost. Constrained decoding is often slower than free generation because the sampler has to mask invalid tokens each step. For latency-critical paths, sometimes free-form generation + robust parsing wins.



6

Observability & continuous improvement:

Log every parse failure with the raw output, and periodically review — often failures reveal schema gaps (a new report type, a new company role we didn’t anticipate). The eval loop informs the schema, not just the prompt.



Key Takeaway



Guarantee shape with schema + Pydantic.



Validate meaning with business rules.



Observe, learn, and evolve the schema.

FOLLOW-UPS

Q: The LLM keeps returning a geography that isn't in your enum. What do you do?

Log it, look at the raw text. Two possibilities: model hallucination (retry with a stricter prompt), or the taxonomy is inPydantic (add the geography if valid). Never silently accept invalid values.

Q: Difference between OpenAI's JSON mode and structured outputs (strict schema)?

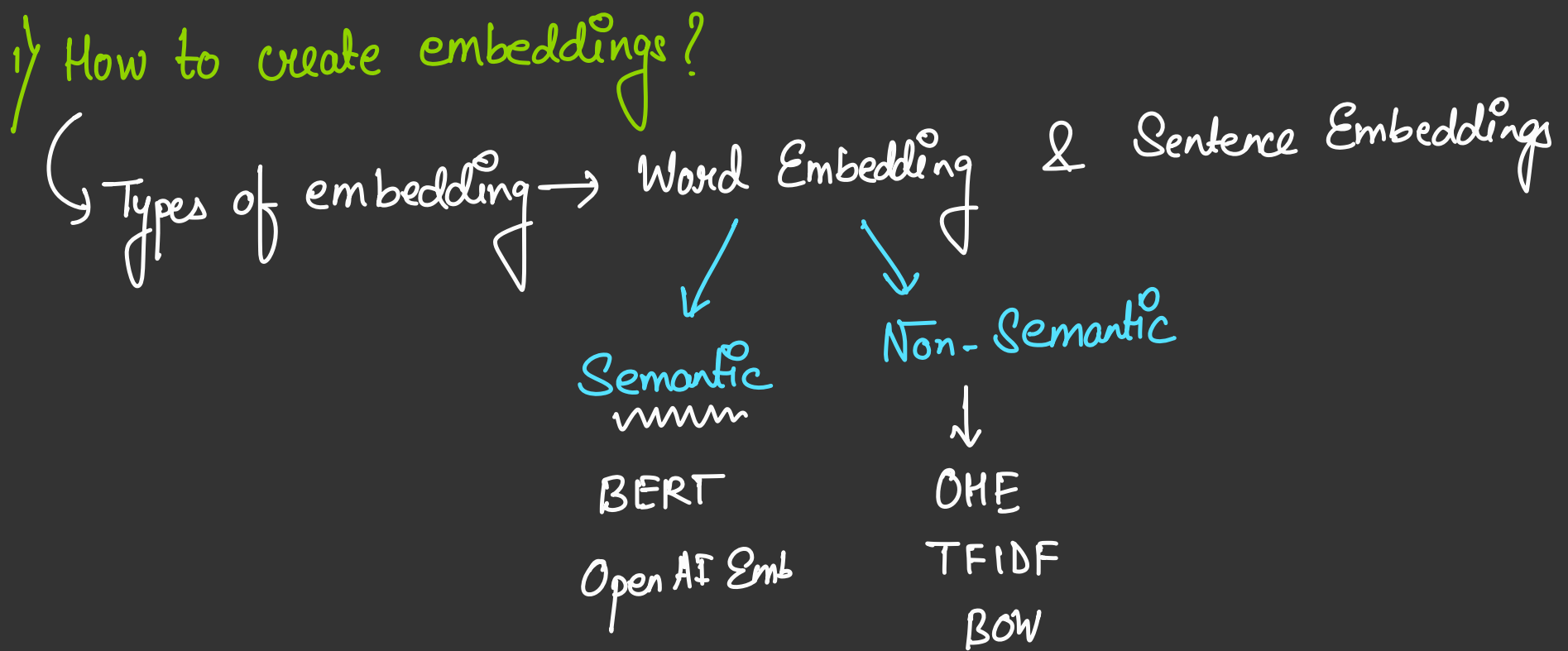
JSON mode guarantees valid JSON syntax but not schema conformance — you can still get missing fields or wrong types. Structured outputs with strict: true constrains generation to your exact schema; guaranteed to conform.

Q: How does Pydantic help beyond just json.loads?

Type coercion (converts "2024" to int if the field is int), validation on ranges, custom validators, enum enforcement, nested model validation, and readable error messages. json.loads gives you a dict — Pydantic gives you a validated typed object.

Embeddings

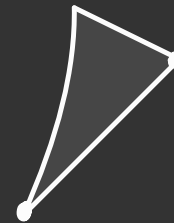
1) How to create embeddings?



Bank
mm
→ [- - - - -]

Apple
mm
→ []

"text-embeddings-3-large"

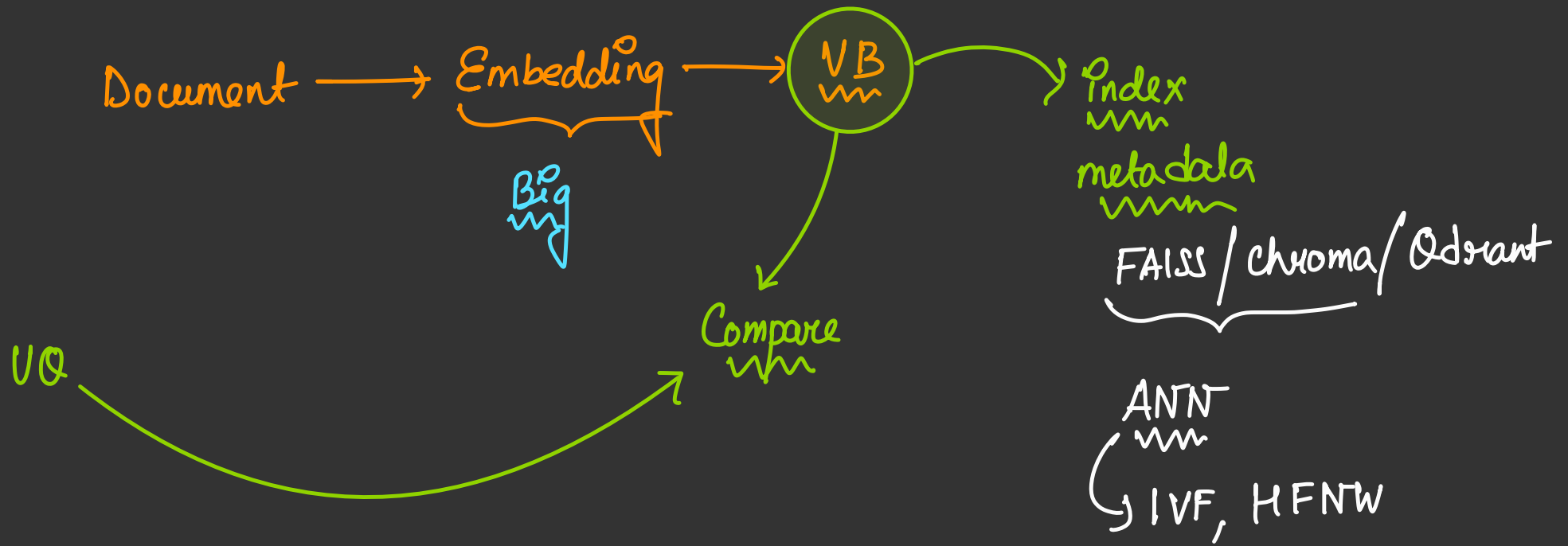


cosine sim
~~~~~  
→  $L_2$  Norm ? (Higher dim) X  
~~~~~

What is the right dimension?
512 / 768
~~~~~

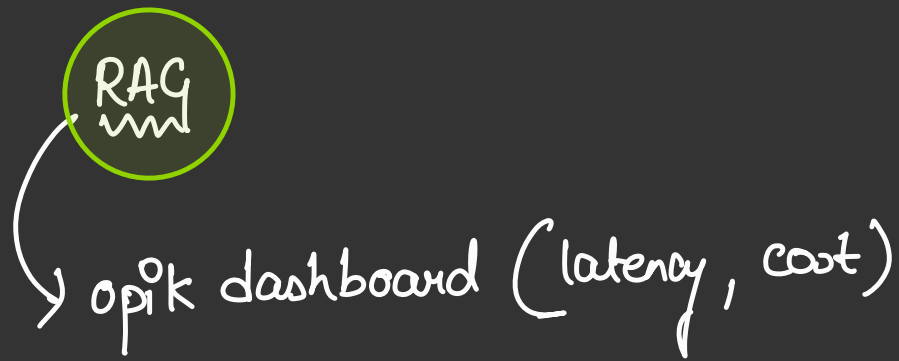
|           | large dim | Vs | Small dim |
|-----------|-----------|----|-----------|
| memory    | more      |    | less      |
| cost      | more      |    | less      |
| precision | more      |    | less      |
| latency   | more      |    | less      |





- 1) ANN (Hybrid Search) + Chroma encoder.
- 2) Cost → Medium size em model
- 3) Caching → low latency.

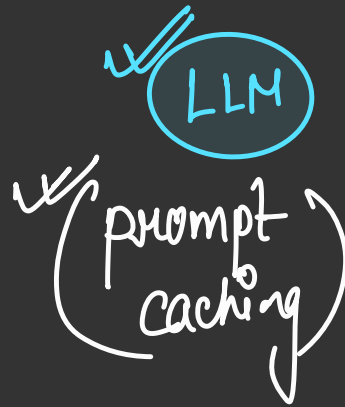
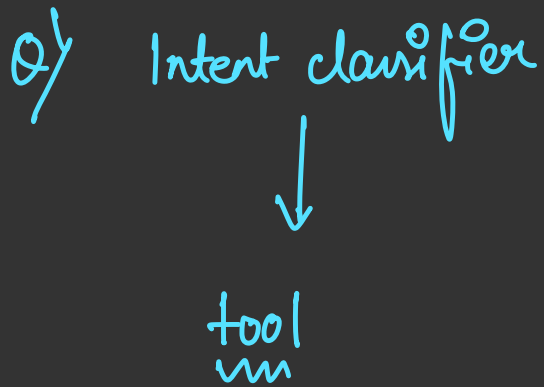
VDB → { text, text vector,  
text metadata



tool

H

$\begin{cases} KV \\ \text{caching} \end{cases}$   
KV (attention)



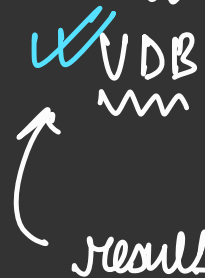
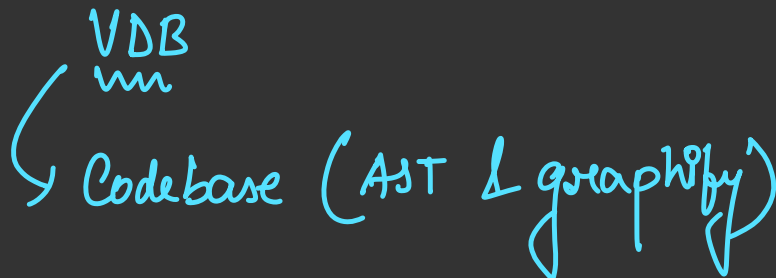
User Query

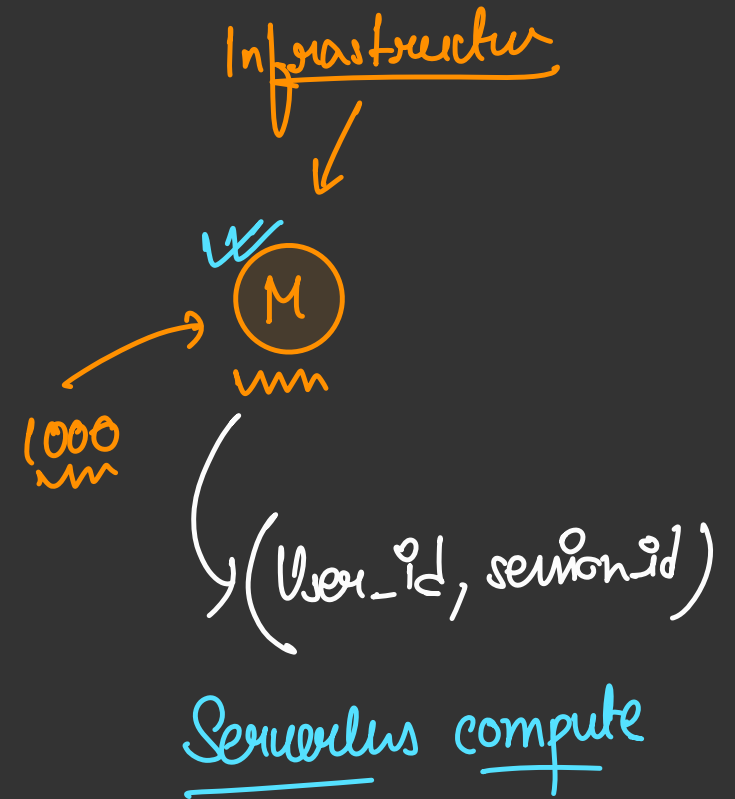
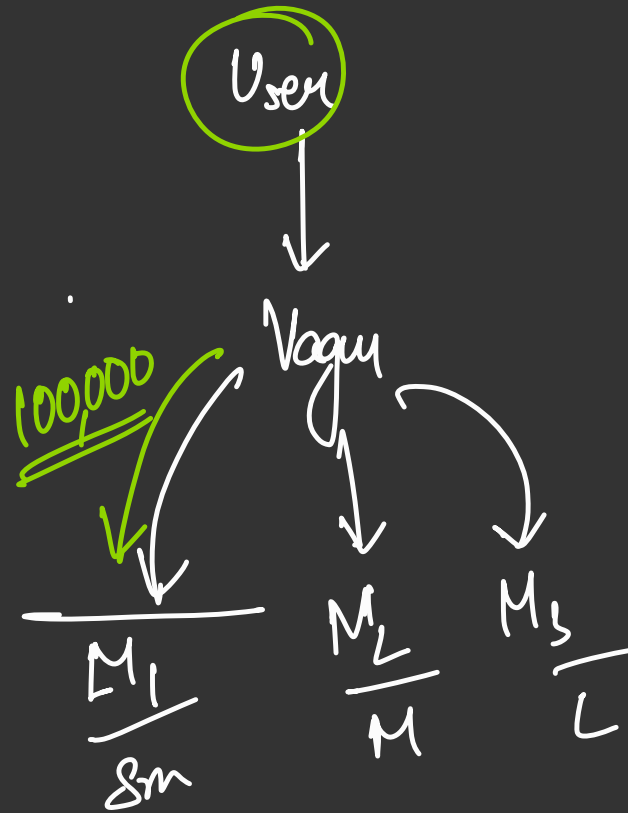
Frontend

Intent Classifier

tool

tool





KV Cache → Attention k & v  
tensor

prompt  
Cache → processed prompt  
prefix

Semantic  
Cache → previous seen  
LLM response